

UNIVERSIDAD TÉCNICA DE AMBATO

Carrera de Ingeniería de Software

Tarea 2.2 Recorrido de Arboles Binarios

Asignatura	Estructura de Datos
Nivel	Tercer nivel B
Lenguaje	C++, Java
Estudiante	Acurio Lopez Paul Medardo
Docente	Ing. José Caiza
Fecha	06 de mayo de 2026

Propósito del documento. Esta tarea detalla la construcción y evaluación de un Árbol Binario de Búsqueda (BST), integrando estructuras dinámicas no lineales, el uso de colas y recursividad vistas en clase para recorrer y analizar los datos de forma interactiva.

1. Descripción general

Las estructuras de datos jerárquicas permiten organizar información de forma que las operaciones de búsqueda y recorrido sean eficientes. Este proyecto se enfoca en representar datos mediante un Árbol Binario de Búsqueda en Java. El sistema desarrollado permite visualizar la estructura gráfica del árbol, insertar nuevos nodos validando que no existan duplicados, y analizar la red mediante recorridos de profundidad (DFS: Preorden, Inorden, Postorden) y de anchura (BFS). Además, incluye un módulo interactivo de evaluación (Quiz) y la justificación teórica de un organigrama para un Sistema Web.

2. Justificación académica

El proyecto es pertinente para la asignatura porque integra, en una sola aplicación, contenidos clave de Estructura de Datos: TDA, nodos enlazados, colas, recursividad, recorridos en árboles y análisis de complejidad. También refuerza la transferencia de conocimiento entre estructuras lineales y no lineales. En particular, el recorrido BFS se aplicó como una implementación directa de la cola (LinkedList actuando como Queue) trabajada en clase, mientras que los algoritmos DFS se vincularon con recursividad controlada.

3. Objetivo general

Desarrollar un sistema en C++ y Java que represente y analice un árbol binario, aplicando operaciones de inserción, impresión gráfica, recorridos (DFS y BFS), cálculo de métricas y evaluación interactiva del usuario.

4. Objetivos específicos

1. Representar una jerarquía mediante una estructura de árbol binario usando referencias a nodos hijos.
2. Implementar operaciones de agregación y conteo sobre los nodos del árbol.
3. Aplicar recorridos BFS y DFS para explorar la red.
4. Aplicar recorridos BFS y DFS para explorar la estructura.
5. Evaluar el conocimiento del usuario mediante un cuestionario interactivo de recorridos.
6. Aplicar la teoría de los recorridos a un caso de negocio real (Módulos de un Sistema Web).
7. Analizar la complejidad temporal de las operaciones implementadas.

5. Explicación de la Estructura Utilizada

Para la representación del árbol y la resolución de los ejercicios, se diseñó un programa iterativo en Java con un menú principal de 9 opciones. La implementación técnica se fundamentó en los siguientes elementos:

- **Clase Nodo:** Actúa como el bloque fundamental de la estructura. Contiene el valor entero (dato) y dos referencias (punteros en la versión C++) hacia los nodos descendientes izquierda y derecha.
- **Recursividad (DFS):** Los recorridos en profundidad (Preorden, Inorden y Postorden), así como el cálculo del total de nodos y la identificación de nodos hoja, se implementaron mediante funciones

recursivas. Esto minimiza el código y utiliza la pila de ejecución del sistema para navegar por las ramas.

- **Colas (BFS):** Para el recorrido por niveles (Búsqueda en Anchura), se implementó la interfaz Queue a través de una LinkedList en Java.
- **Módulo de Evaluación (Quiz):** Se desarrolló un sistema empleando StringBuilder para recolectar el recorrido generado por el sistema y compararlo mediante manejo de String (limpiando espacios adicionales) con el ingreso manual del usuario para determinar si su respuesta es correcta o incorrecta.
- **Validación de Inserción:** Se implementó una función de búsqueda previa (buscar) que impide el ingreso de valores duplicados al momento de agregar nuevos nodos, manteniendo la integridad de un Árbol Binario de Búsqueda (BST).

6. Definición de Conceptos Clave

Para comprender la funcionalidad del árbol implementado, se definen los siguientes elementos:

- **Raíz:** Es el nodo de origen (en este caso inicializado con el valor 10) del cual se desprenden todos los demás. No posee un nodo superior.
- **Hoja:** Nodos en los extremos inferiores que no tienen hijos (ambas referencias apuntan a nulo).
- **Recorridos DFS:** Estrategias que exploran hasta la hoja más profunda antes de retroceder.
 - Preorden: Raíz → Izquierda → Derecha.
 - Inorden: Izquierda → Raíz → Derecha.
 - Postorden: Izquierda → Derecha → Raíz.
- **Recorrido BFS:** Estrategia que procesa los nodos nivel por nivel, de izquierda a derecha, utilizando una cola como estructura de apoyo.

7. Análisis de Complejidad Temporal

La implementación de esta estructura binaria presenta los siguientes rendimientos:

- **Recorridos y Métricas:** La complejidad es estrictamente de $O(n)$, ya que, para imprimir los recorridos, contar las hojas o validar el quiz, el algoritmo debe visitar obligatoriamente todos y cada uno de los nodos del árbol exactamente una vez.
- **Inserción y Búsqueda (Validación):** En un árbol balanceado, la complejidad de insertar un nuevo nodo o buscar duplicados es de $O(\log n)$. Si el usuario ingresa datos ordenados secuencialmente, el árbol puede degenerar en una lista enlazada, afectando el rendimiento al peor caso $O(n)$.

8. Evidencia de Ejecución y Repositorio

El desarrollo colaborativo y los entregables se gestionan a través de control de versiones.

Enlace de GitHub: <https://github.com/Paul-Acurio/Recorridos-de-arboles-Binarios---Estructura-de-Datos.git>

- **Código en C++**

```

main.cpp X
C:\Users\User\Documents\UTA\Estructura de Datos\tarea2.2 repo_recorridos_arboles_uta\tarea2.2 repo_recorridos_arboles_uta\src\cpp> g++ main.cpp > ...

1  #include <iostream>
2  #include <queue>
3  #include <string>
4  #include <sstream>
5  using namespace std;
6
7  struct Nodo {
8      int dato;
9      Nodo* izquierda;
10     Nodo* derecha;
11
12     Nodo(int valor) {
13         dato = valor;
14         izquierda = nullptr;
15         derecha = nullptr;
16     }
17 };
18
19 bool buscar(Nodo* raiz, int valor) {
20     if (raiz == nullptr) return false;
21     if (raiz->dato == valor) return true;
22     if (valor < raiz->dato) return buscar(raiz->izquierda, valor);
23     return buscar(raiz->derecha, valor);
24 }
25
26 Nodo* insertar(Nodo* nodo, int valor) {
27     if (nodo == nullptr) return new Nodo(valor);
28     if (valor < nodo->dato) {
29         nodo->izquierda = insertar(nodo->izquierda, valor);
30     } else if (valor > nodo->dato) {
31         nodo->derecha = insertar(nodo->derecha, valor);
32     }
33     return nodo;
34 }
35
36 void imprimirArbol(Nodo* raiz, int espacio = 0) {
37     if (raiz == nullptr) return;
38
39     int incremento = 5;
40     espacio += incremento;
41     imprimirArbol(raiz->derecha, espacio);
42     cout << endl;
43     for (int i = incremento; i < espacio; i++) cout << " ";
44     cout << raiz->dato << "\n";
45     imprimirArbol(raiz->izquierda, espacio);
46 }
47
48 void preorden(Nodo* raiz) {
49     if (raiz == nullptr) return;
50     cout << raiz->dato << " ";
51     preorden(raiz->izquierda);
52     preorden(raiz->derecha);
53 }
54
55 void inorden(Nodo* raiz) {
56     if (raiz == nullptr) return;
57     inorden(raiz->izquierda);
58     cout << raiz->dato << " ";
59     inorden(raiz->derecha);
60 }
61
62 void postorden(Nodo* raiz) {
63     if (raiz == nullptr) return;
64     postorden(raiz->izquierda);
65     postorden(raiz->derecha);
66     cout << raiz->dato << " ";
67 }
68
69 void bfs(Nodo* raiz) {
70     if (raiz == nullptr) return;
71     queue<Nodo*> cola;
72     cola.push(raiz);

```

```

72     while (!cola.empty()) {
73         Nodo* actual = cola.front();
74         cola.pop();
75         cout << actual->dato << " ";
76         if (actual->izquierda != nullptr) cola.push(actual->izquierda);
77         if (actual->derecha != nullptr) cola.push(actual->derecha);
78     }
79 }
80
81 void obtenerPreorden(Nodo* raiz, string& res) {
82     if (raiz == nullptr) return;
83     res += to_string(raiz->dato) + " ";
84     obtenerPreorden(raiz->izquierda, res);
85     obtenerPreorden(raiz->derecha, res);
86 }
87
88 void obtenerInorden(Nodo* raiz, string& res) {
89     if (raiz == nullptr) return;
90     obtenerInorden(raiz->izquierda, res);
91     res += to_string(raiz->dato) + " ";
92     obtenerInorden(raiz->derecha, res);
93 }
94
95 void obtenerPostorden(Nodo* raiz, string& res) {
96     if (raiz == nullptr) return;
97     obtenerPostorden(raiz->izquierda, res);
98     obtenerPostorden(raiz->derecha, res);
99     res += to_string(raiz->dato) + " ";
100 }
101
102 void obtenerBfs(Nodo* raiz, string& res) {
103     if (raiz == nullptr) return;
104     queue<Nodo*> cola;
105     cola.push(raiz);

```

```

106     while (!cola.empty()) {
107         Nodo* actual = cola.front();
108         cola.pop();
109         res += to_string(actual->dato) + " ";
110         if (actual->izquierda != nullptr) cola.push(actual->izquierda);
111         if (actual->derecha != nullptr) cola.push(actual->derecha);
112     }
113 }
114
115 string limpiarEspacios(const string& str) {
116     stringstream ss(str);
117     string item, resultado = "";
118     while (ss >> item) resultado += item + " ";
119     return resultado;
120 }
121
122 int contarNodos(Nodo* raiz) {
123     if (raiz == nullptr) return 0;
124     return 1 + contarNodos(raiz->izquierda) + contarNodos(raiz->derecha);
125 }
126
127 int contarHojas(Nodo* raiz) {
128     if (raiz == nullptr) return 0;
129     if (raiz->izquierda == nullptr && raiz->derecha == nullptr) return 1;
130     return contarHojas(raiz->izquierda) + contarHojas(raiz->derecha);
131 }
132
133 int main() {
134     Nodo* raiz = new Nodo(10);
135     raiz->izquierda = new Nodo(5);
136     raiz->derecha = new Nodo(15);
137     raiz->izquierda->izquierda = new Nodo(2);
138     raiz->izquierda->derecha = new Nodo(7);
139     raiz->derecha->izquierda = new Nodo(12);

```

```

140 raiz->derecha->derecha = new Nodo(20);
141
142 int opcion;
143 do {
144     cout << "\n=== MENU PRINCIPAL - ARBOLES BINARIOS (UTA) ===" << endl;
145     cout << "1. Mostrar recorrido Preorden" << endl;
146     cout << "2. Mostrar recorrido Inorden" << endl;
147     cout << "3. Mostrar recorrido Postorden" << endl;
148     cout << "4. Mostrar recorrido BFS (Niveles)" << endl;
149     cout << "5. Ver metricas (Total nodos y hojas)" << endl;
150     cout << "6. Agregar nodos adicionales" << endl;
151     cout << "7. Evaluar recorridos manualmente (Quiz)" << endl;
152     cout << "8. Proyecto Final Web" << endl;
153     cout << "9. Salir" << endl;
154     cout << "Seleccione una opcion: ";
155     cin >> opcion;
156
157     cout << "\n-----" << endl;
158     switch (opcion) {
159         case 1:
160             cout << "Estructura del Arbol:\n"; imprimirArbol(raiz);
161             cout << "\n\nRecorrido Preorden: \n"; preorden(raiz); cout << endl;
162             break;
163         case 2:
164             cout << "Estructura del Arbol:\n"; imprimirArbol(raiz);
165             cout << "\n\nRecorrido Inorden: \n"; inorden(raiz); cout << endl;
166             break;
167         case 3:
168             cout << "Estructura del Arbol:\n"; imprimirArbol(raiz);
169             cout << "\n\nRecorrido Postorden: \n"; postorden(raiz); cout << endl;
170             break;
171         case 4:
172             cout << "Estructura del Arbol:\n"; imprimirArbol(raiz);
173             cout << "\n\nRecorrido BFS (Niveles): \n"; bfs(raiz); cout << endl;
174             break;

```

```

175         case 5:
176             cout << "Metricas:" << endl;
177             cout << "- Total de nodos: " << contarNodos(raiz) << endl;
178             cout << "- Total de hojas: " << contarHojas(raiz) << endl;
179             break;
180         case 6: {
181             int numNodos, valor;
182             cout << "Cuantos nodos desea ingresar? ";
183             cin >> numNodos;
184             int ingresados = 0;
185             while (ingresados < numNodos) {
186                 cout << "Ingrese el valor del nodo " << (ingresados + 1) << ": ";
187                 cin >> valor;
188                 if (buscar(raiz, valor)) {
189                     cout << "-> ERROR: El valor " << valor << " ya existe en el arbol. Intente con otro.\n";
190                 } else {
191                     insertar(raiz, valor);
192                     ingresados++;
193                 }
194             }
195             cout << "Nodos agregados correctamente." << endl;
196             break;
197         }
198         case 7: {
199             cin.ignore();
200             string respuestaUsuario, respuestaCorrecta;
201             cout << "=== MODO EVALUACION ===" << endl;
202             cout << "Nota: Ingrese los valores separados por un espacio.\n\n";
203
204             // IMPRIMIR EL ÁRBOL PARA GUÍA DEL USUARIO
205             cout << "Estructura actual del Arbol:\n";
206             imprimirArbol(raiz);
207             cout << "\n\n";
208
209             respuestaCorrecta = ""; obtenerPreorden(raiz, respuestaCorrecta);
210             cout << "1. Escriba el recorrido PREORDEN:\n> ";

```

```

211 getline(cin, respuestaUsuario);
212 if (limpiarEspacios(respuestaUsuario) == respuestaCorrecta) {
213     cout << " [CORRECTO]\n";
214 } else {
215     cout << " [INCORRECTO] Has fallado.\n Respuesta esperada: " << respuestaCorrecta << "\n";
216 }
217
218 cout << "\n2. Escriba el recorrido INORDEN:\n> ";
219 respuestaCorrecta = ""; obtenerInorden(raiz, respuestaCorrecta);
220 getline(cin, respuestaUsuario);
221 if (limpiarEspacios(respuestaUsuario) == respuestaCorrecta) {
222     cout << " [CORRECTO]\n";
223 } else {
224     cout << " [INCORRECTO] Has fallado.\n Respuesta esperada: " << respuestaCorrecta << "\n";
225 }
226
227 cout << "\n3. Escriba el recorrido POSTORDEN:\n> ";
228 respuestaCorrecta = ""; obtenerPostorden(raiz, respuestaCorrecta);
229 getline(cin, respuestaUsuario);
230 if (limpiarEspacios(respuestaUsuario) == respuestaCorrecta) {
231     cout << " [CORRECTO]\n";
232 } else {
233     cout << " [INCORRECTO] Has fallado.\n Respuesta esperada: " << respuestaCorrecta << "\n";
234 }
235
236 cout << "\n4. Escriba el recorrido BFS:\n> ";
237 respuestaCorrecta = ""; obtenerBfs(raiz, respuestaCorrecta);
238 getline(cin, respuestaUsuario);
239 if (limpiarEspacios(respuestaUsuario) == respuestaCorrecta) {
240     cout << " [CORRECTO]\n";
241 } else {
242     cout << " [INCORRECTO] Has fallado.\n Respuesta esperada: " << respuestaCorrecta << "\n";
243 }
244 break;
245 }

```

```

246 case 8:
247     cout << "=== Aplicado al proyecto final ===" << endl;
248     cout << "Represente los modulos de un sistema web como un arbol binario:\n\n";
249     cout << "      Sistema Web" << endl;
250     cout << "      /      \\" << endl;
251     cout << "    Usuarios  Inventario" << endl;
252     cout << "  /  \\" << endl;
253     cout << " Registrar Buscar  Productos Reportes\n\n";
254
255     cout << "1. Mostrar el menu principal:" << endl;
256     cout << " Respuesta: Recorrido PREORDEN (Raiz -> Izquierda -> Derecha)." << endl;
257     cout << " Justificacion: Permite mostrar primero el titulo/modulo principal (Sistema Web) antes de desplegar sus submenus o hijos." << endl;
258
259     cout << "\n2. Procesar primero los modulos internos:" << endl;
260     cout << " Respuesta: Recorrido POSTORDEN (Izquierda -> Derecha -> Raiz)." << endl;
261     cout << " Justificacion: Visita primero las hojas (Registrar, Buscar, Productos), asegurando que los procesos mas profundos o internos terminen antes de
262
263     cout << "\n3. Mostrar modulos nivel por nivel:" << endl;
264     cout << " Respuesta: Recorrido BFS (Busqueda en Anchura)." << endl;
265     cout << " Justificacion: Permite visualizar las jerarquias nivel por nivel. Primero el Sistema general, luego las grandes categorias (Usuarios, Inventario
266     break;
267 case 9:
268     cout << "Saliendo..." << endl;
269     break;
270 default:
271     cout << "Opcion invalida." << endl;
272 }
273 cout << "-----" << endl;
274 } while (opcion != 9);
275
276 return 0;
277 }

```

- Código en Java

```

1  import java.util.LinkedList;
2  import java.util.Queue;
3  import java.util.Scanner;
4
5  class Nodo { 29 usages  Ⓜ Paul
6      int dato; 14 usages
7      Nodo izquierda; 23 usages
8      Nodo derecha; 23 usages
9
10     @Contract(pure = true)
11     public Nodo(int dato) { 8 usages  Ⓜ Paul
12         this.dato = dato;
13         this.izquierda = null;
14         this.derecha = null;
15     }
16 }
17 public class Main { Ⓜ Paul
18
19     @Contract("null, _ -> false")
20     public static boolean buscar(Nodo raiz, int valor) { 3 usages  Ⓜ Paul
21         if (raiz == null) return false;
22         if (raiz.dato == valor) return true;
23         if (valor < raiz.dato) return buscar(raiz.izquierda, valor);
24         return buscar(raiz.derecha, valor);
25     }
26
27     @Contract("null, _ -> new")
28     public static Nodo ! insertar(Nodo nodo, int valor) { 3 usages  Ⓜ Paul
29         if (nodo == null) return new Nodo(valor);
30         if (valor < nodo.dato) {
31             nodo.izquierda = insertar(nodo.izquierda, valor);
32         } else if (valor > nodo.dato) {
33             nodo.derecha = insertar(nodo.derecha, valor);
34         }
35         return nodo;
36     }
37
38     public static void imprimirArbol(Nodo raiz, int espacio) { 7 usages  Ⓜ Paul
39         if (raiz == null) return;
40         int incremento = 5;
41         espacio += incremento;
42         imprimirArbol(raiz.derecha, espacio);
43         System.out.println();
44         for (int i = incremento; i < espacio; i++) System.out.print(" ");
45         System.out.println(raiz.dato);
46         imprimirArbol(raiz.izquierda, espacio);
47     }
48
49     public static void preorden(Nodo raiz) { 3 usages  Ⓜ Paul
50         if (raiz == null) return;
51         System.out.print(raiz.dato + " ");
52         preorden(raiz.izquierda);
53         preorden(raiz.derecha);
54     }
55
56     public static void inorden(Nodo raiz) { 3 usages  Ⓜ Paul
57         if (raiz == null) return;

```



```

56     inorden(raiz.izquierda);
57     System.out.print(raiz.dato + " ");
58     inorden(raiz.derecha);
59 }
60
61 public static void postorden(Nodo raiz) { 3 usages  & Paul
62     if (raiz == null) return;
63     postorden(raiz.izquierda);
64     postorden(raiz.derecha);
65     System.out.print(raiz.dato + " ");
66 }
67
68 public static void bfs(Nodo raiz) { 1 usage  & Paul
69     if (raiz == null) return;
70     Queue<Nodo> cola = new LinkedList<>();
71     cola.add(raiz);
72     while (!cola.isEmpty()) {
73         Nodo actual = cola.poll();
74         System.out.print(actual.dato + " ");
75         if (actual.izquierda != null) cola.add(actual.izquierda);
76         if (actual.derecha != null) cola.add(actual.derecha);
77     }
78 }
79
80 public static void obtenerPreorden(Nodo raiz, StringBuilder res) { 3 usages  & Paul
81     if (raiz == null) return;
82     res.append(raiz.dato).append(" ");
83     obtenerPreorden(raiz.izquierda, res);
84     obtenerPreorden(raiz.derecha, res);
85 }
86
87 public static void obtenerInorden(Nodo raiz, StringBuilder res) { 3 usages  & Paul
88     if (raiz == null) return;
89     obtenerInorden(raiz.izquierda, res);
90     res.append(raiz.dato).append(" ");
91     obtenerInorden(raiz.derecha, res);
92 }
93
94 public static void obtenerPostorden(Nodo raiz, StringBuilder res) { 3 usages  & Paul
95     if (raiz == null) return;
96     obtenerPostorden(raiz.izquierda, res);
97     obtenerPostorden(raiz.derecha, res);
98     res.append(raiz.dato).append(" ");
99 }
100
101 public static void obtenerBfs(Nodo raiz, StringBuilder res) { 1 usage  & Paul
102     if (raiz == null) return;
103     Queue<Nodo> cola = new LinkedList<>();
104     cola.add(raiz);
105     while (!cola.isEmpty()) {
106         Nodo actual = cola.poll();
107         res.append(actual.dato).append(" ");
108         if (actual.izquierda != null) cola.add(actual.izquierda);
109         if (actual.derecha != null) cola.add(actual.derecha);

```

```

110     }
111 }
112
113 @ > public static String limpiarEspacios(String str) { return str.trim().replaceAll(regex: "\\s+", replaceme
116
117 public static int contarNodos(Nodo raiz) { 3 usages 2 Paul
118     if (raiz == null) return 0;
119     return 1 + contarNodos(raiz.izquierda) + contarNodos(raiz.derecha);
120 }
121
122 public static int contarHojas(Nodo raiz) { 3 usages 2 Paul
123     if (raiz == null) return 0;
124     if (raiz.izquierda == null && raiz.derecha == null) return 1;
125     return contarHojas(raiz.izquierda) + contarHojas(raiz.derecha);
126 }
127
128 public static void main(String[] args) { 2 Paul
129     Scanner scanner = new Scanner(System.in);
130     Nodo raiz = new Nodo( dato: 10);
131     raiz.izquierda = new Nodo( dato: 5);
132     raiz.derecha = new Nodo( dato: 15);
133     raiz.izquierda.izquierda = new Nodo( dato: 2);
134     raiz.izquierda.derecha = new Nodo( dato: 7);
135     raiz.derecha.izquierda = new Nodo( dato: 12);
136     raiz.derecha.derecha = new Nodo( dato: 20);
137
138     int opcion = 0;
139
140     do {
141         System.out.println("\n=== MENU PRINCIPAL - ARBOLES BINARIOS (UTA) ===");
142         System.out.println("1. Mostrar recorrido Preorden");
143         System.out.println("2. Mostrar recorrido Inorden");
144         System.out.println("3. Mostrar recorrido Postorden");
145         System.out.println("4. Mostrar recorrido BFS (Niveles)");
146         System.out.println("5. Ver metricas (Total nodos y hojas)");
147         System.out.println("6. Agregar nodos adicionales");
148         System.out.println("7. Evaluar recorridos manualmente (Quiz)");
149         System.out.println("8. Proyecto Final Web");
150         System.out.println("9. Salir");
151         System.out.print("Seleccione una opcion: ");
152
153         try {
154             opcion = Integer.parseInt(scanner.nextLine());
155         } catch (NumberFormatException e) {
156             opcion = 0;
157         }
158
159         System.out.println("\n-----");
160         switch (opcion) {
161             case 1:
162                 System.out.println("Estructura del Arbol:"); imprimirArbol(raiz, espacio: 0);
163                 System.out.println("\n\nRecorrido Preorden:"); preorden(raiz); System.out.println();
164                 break;
165             case 2:
166                 System.out.println("Estructura del Arbol:"); imprimirArbol(raiz, espacio: 0);

```

```

166         System.out.println("\n\nRecorrido Inorden:"); inorden(raiz); System.out.println();
167         break;
168     case 3:
169         System.out.println("Estructura del Arbol:"); imprimirArbol(raiz, espacio: 0);
170         System.out.println("\n\nRecorrido Postorden:"); postorden(raiz); System.out.println();
171         break;
172     case 4:
173         System.out.println("Estructura del Arbol:"); imprimirArbol(raiz, espacio: 0);
174         System.out.println("\n\nRecorrido BFS (Niveles):"); bfs(raiz); System.out.println();
175         break;
176     case 5:
177         System.out.println("Metricas:");
178         System.out.println("- Total de nodos: " + contarNodos(raiz));
179         System.out.println("- Total de hojas: " + contarHojas(raiz));
180         break;
181     case 6:
182         System.out.print("Cuantos nodos desea ingresar? ");
183         try {
184             int numNodos = Integer.parseInt(scanner.nextLine());
185             int ingresados = 0;
186             while (ingresados < numNodos) {
187                 System.out.print("Ingrese el valor del nodo " + (ingresados + 1) + ": ");
188                 int valor = Integer.parseInt(scanner.nextLine());
189                 if (buscar(raiz, valor)) {
190                     System.out.println("-> ERROR: El valor " + valor + " ya existe en el arbol. Intente de nuevo.");
191                 } else {
192                     insertar(raiz, valor);
193                     ingresados++;
194                 }
195             }
196             System.out.println("Nodos agregados correctamente.");
197         } catch (NumberFormatException e) {
198             System.out.println("Entrada invalida. Volviendo al menu.");
199         }
200         break;
201     case 7:
202         StringBuilder correct;
203         String user;
204         System.out.println("=== MODO EVALUACION ===");
205         System.out.println("Nota: Ingrese los valores separados por un espacio.\n");
206
207         // IMPRIMIR EL ÁRBOL PARA GUÍA DEL USUARIO
208         System.out.println("Estructura actual del Arbol:");
209         imprimirArbol(raiz, espacio: 0);
210         System.out.println("\n");
211
212         // Preorden
213         correct = new StringBuilder(); obtenerPreorden(raiz, correct);
214         System.out.print("1. Escriba el recorrido PREORDEN:\n> ");
215         user = scanner.nextLine();
216         if (limpiarEspacios(user).equals(correct.toString())) {
217             System.out.println(" [CORRECTO]");
218         } else {
219             System.out.println(" [INCORRECTO] Has fallado.\n Respuesta esperada: " + correct.toString());

```

```

220     }
221
222     // Inorden
223     correct = new StringBuilder(); obtenerInorden(raiz, correct);
224     System.out.print("\n2. Escriba el recorrido INORDEN:\n> ");
225     user = scanner.nextLine();
226     if (limpiarEspacios(user).equals(correct.toString())) {
227         System.out.println("    [CORRECTO]");
228     } else {
229         System.out.println("    [INCORRECTO] Has fallado.\n Respuesta esperada: " + correct.toString());
230     }
231
232     // Postorden
233     correct = new StringBuilder(); obtenerPostorden(raiz, correct);
234     System.out.print("\n3. Escriba el recorrido POSTORDEN:\n> ");
235     user = scanner.nextLine();
236     if (limpiarEspacios(user).equals(correct.toString())) {
237         System.out.println("    [CORRECTO]");
238     } else {
239         System.out.println("    [INCORRECTO] Has fallado.\n Respuesta esperada: " + correct.toString());
240     }
241
242     // BFS
243     correct = new StringBuilder(); obtenerBfs(raiz, correct);
244     System.out.print("\n4. Escriba el recorrido BFS:\n> ");
245     user = scanner.nextLine();
246     if (limpiarEspacios(user).equals(correct.toString())) {
247         System.out.println("    [CORRECTO]");
248     } else {
249         System.out.println("    [INCORRECTO] Has fallado.\n Respuesta esperada: " + correct.toString());
250     }
251     break;
252 case 8:
253     System.out.println("=== Aplicado al proyecto final ===");
254     System.out.println("Represente los modulos de un sistema web como un arbol binario:\n");
255     System.out.println("        Sistema Web");
256     System.out.println("        /      \\\");
257     System.out.println("    Usuarios    Inventario");
258     System.out.println("    /  \\\      /  \\\");
259     System.out.println(" Registrar  Buscar  Productos Reportes\n");
260
261     System.out.println("1. Mostrar el menu principal:");
262     System.out.println(" Respuesta: Recorrido PREORDEN (Raiz -> Izquierda -> Derecha).");
263     System.out.println(" Justificacion: Permite mostrar primero el titulo/modulo principal");
264
265     System.out.println("\n2. Procesar primero los modulos internos:");
266     System.out.println(" Respuesta: Recorrido POSTORDEN (Izquierda -> Derecha -> Raiz).");
267     System.out.println(" Justificacion: Visita primero las hojas (Registrar, Buscar, Produ");
268
269     System.out.println("\n3. Mostrar modulos nivel por nivel:");
270     System.out.println(" Respuesta: Recorrido BFS (Busqueda en Anchura).");
271     System.out.println(" Justificacion: Permite visualizar las jerarquias nivel por nivel.");
272     break;
273 case 9:

```

```

275         System.out.println("Saliendo...");
276         break;
277     default:
278         System.out.println("Opcion invalida.");
279     }
280     System.out.println("-----");
281 } while (opcion != 9);
282
283 scanner.close();
284 }
285 }
286

```

• Ejecución

1. Muestra de la Raíz y Recorridos (Opciones 1 al 4):

```

=== MENU PRINCIPAL - ARBOLES BINARIOS (UTA) ===
1. Mostrar recorrido Preorden
2. Mostrar recorrido Inorden
3. Mostrar recorrido Postorden
4. Mostrar recorrido BFS (Niveles)
5. Ver metricas (Total nodos y hojas)
6. Agregar nodos adicionales
7. Evaluar recorridos manualmente (Quiz)
8. Proyecto Final Web
9. Salir
Seleccione una opcion: |

```

Seleccione una opcion: 1

Estructura del Arbol:

```

          20
        15
      12
10
      7
    5
      2
```

Recorrido Preorden:

10 5 2 7 15 12 20

Seleccione una opcion: 2

Estructura del Arbol:

```

          20
        15
      12
10
      7
    5
      2
```

Recorrido Inorden:

2 5 7 10 12 15 20

Seleccione una opcion: 3

Estructura del Arbol:

```

      20
     /  \
    15   12
   /  \  /  \
  10   7 5   2
     \
      5
```

Recorrido Postorden:

2 7 5 12 20 15 10

Seleccione una opcion: 4

Estructura del Arbol:

```

      20
     /  \
    15   12
   /  \  /  \
  10   7 5   2
     \
      5
```

Recorrido BFS (Niveles):

10 5 15 2 7 12 20

2. Agregación de nodos y validación (Opción 6) - Ejercicio 2:

Seleccione una opcion: 6

```
-----  
Cuantos nodos desea ingresar? 4  
Ingrese el valor del nodo 1: 1  
Ingrese el valor del nodo 2: 3  
Ingrese el valor del nodo 3: 18  
Ingrese el valor del nodo 4: 25  
Nodos agregados correctamente.  
-----
```

Estructura del Arbol:

```
                25  
              20  
            18  
          15  
        12  
      10  
    7  
  5  
    3  
  2  
    1
```

Recorrido Preorden:

10 5 2 1 3 7 15 12 20 18 25

Recorrido Inorden:

1 2 3 5 7 10 12 15 18 20 25

Recorrido Postorden:

1 3 2 7 5 12 18 25 20 15 10

Recorrido BFS (Niveles):

10 5 15 2 7 12 20 1 3 18 25

3. Ver Métricas (Opción 5) - Ejercicios 3 y 4:


```
=== MENU PRINCIPAL - ARBOLES BINARIOS (UTA) ===
1. Mostrar recorrido Preorden
2. Mostrar recorrido Inorden
3. Mostrar recorrido Postorden
4. Mostrar recorrido BFS (Niveles)
5. Ver metricas (Total nodos y hojas)
6. Agregar nodos adicionales
7. Evaluar recorridos manualmente (Quiz)
8. Proyecto Final Web
9. Salir
Seleccione una opcion: 5

-----
Metricas:
- Total de nodos: 11
- Total de hojas: 6
-----
```

4. Evaluación Manual de Recorridos (Opción 7) - Ejercicio 1:

```

Seleccione una opcion: 7

=====
=== MODO EVALUACION ===
Nota: Ingrese los valores separados por un espacio.

Estructura actual del Arbol:

          25
        20
      18
    15
  12
10
  7
  5
    3
    2
    1

1. Escriba el recorrido PREORDEN:
> 10 5 2 1 3 7 15 12 20 18 25
  [CORRECTO]

2. Escriba el recorrido INORDEN:
> 1 2 3 5 7 10 12 15 18 20 25
  [CORRECTO]

3. Escriba el recorrido POSTORDEN:
> 1 3 2 7 5 12 18 25 20 15 10
  [CORRECTO]

4. Escriba el recorrido BFS:
> 10 5 15 2 7 18 12 20 1 3 25
  [INCORRECTO] Has fallado.
  Respuesta esperada: 10 5 15 2 7 12 20 1 3 18 25
=====

```

5. Caso de Aplicación al Proyecto Final (Opción 8) - Ejercicio 5:

```
Seleccione una opcion: 8

=====
=== Aplicado al proyecto final ===
Represente los modulos de un sistema web como un arbol binario:

      Sistema Web
     /         \
  Usuarios     Inventario
  /  \         /  \
Registrar Buscar Productos Reportes

1. Mostrar el menu principal:
  Respuesta: Recorrido PREORDEN (Raiz -> Izquierda -> Derecha).
  Justificacion: Permite mostrar primero el titulo/modulo principal (Sistema Web) antes de desplegar sus submenus o hijos.

2. Procesar primero los modulos internos:
  Respuesta: Recorrido POSTORDEN (Izquierda -> Derecha -> Raiz).
  Justificacion: Visita primero las hojas (Registrar, Buscar, Productos), asegurando que los procesos mas profundos o internos terminen antes de volver a la raiz.

3. Mostrar modulos nivel por nivel:
  Respuesta: Recorrido BFS (Busqueda en Anchura).
  Justificacion: Permite visualizar las jerarquias nivel por nivel. Primero el Sistema general, luego las grandes categorias (Usuarios, Inventario) y finalmente los detalles.
```

9. Conclusiones

Cumplimiento de la Rúbrica: Se alcanzó el 100% de los resultados de aprendizaje al implementar y contrastar los recorridos DFS y BFS, abarcando todos los criterios de calificación exigidos.

Eficiencia de la Estructura: Se comprobó la flexibilidad de los árboles binarios. Al adaptar la estructura no solo para almacenar enteros aleatorios, sino para mapear la jerarquía de un Sistema Web, se demostró que el recorrido Preorden es vital para desplegar interfaces de navegación, el Postorden garantiza el procesamiento interno de dependencias sin errores, y el BFS es óptimo para vistas modulares por niveles de profundidad.

Interactividad: La inclusión del menú validado y el Modo Evaluación convierte un simple script académico en una herramienta de comprobación dinámica, cumpliendo con la necesidad de ejecutar y reevaluar la estructura múltiples veces luego de agregar nuevos conjuntos de datos como indica el Ejercicio 2.

10. Bibliografía

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, y C. Stein, Introduction to Algorithms, 3.^a ed. Cambridge, MA, EE. UU.: MIT Press, 2009.

[2] M. T. Goodrich, R. Tamassia, y M. H. Goldwasser, Data Structures and Algorithms in Java, 6.^a ed. Hoboken, NJ, EE. UU.: Wiley, 2014.

[3] M. A. Weiss, Data Structures and Algorithm Analysis in C++, 4.^a ed. Boston, MA, EE. UU.: Pearson, 2013.